Anson

2 Jan 2026

RevoU FSDA Batch OCT25

Link Google Colab

Any identifier (eg. client ID, card ID) should be converted to string

```
# 1. Convert identifier columns to string type
id_cols = ['id', 'client_id', 'card_number', 'cvv']
df_card_v1[id_cols] = df_card_v1[id_cols].astype(str)
```

All count data should be converted to integer

```
# 2. Convert count columns to integer
count_cols = ['count_nonfraud_trx_L6M', 'count_fraud_trx_L6M']
df_card_v1[count_cols] = df_card_v1[count_cols].astype('Int64')
```

All data related to money should be in float type (by default, since the column in data contains 'Rp' with thousand separator, the dtype is string.

```
# 3. Clean money columns: remove 'Rp' and '.', then convert to float
money_cols = ['credit_limit', 'amt_nonfraud_trx_L6M', 'amt_fraud_trx_L6M']
for col in money_cols:
    # Ensure the column is treated as string for replacement, then clean
    df_card_v1[col] = df_card_v1[col].astype(str).str.replace('Rp', '', regex=False).str.replace('.',
'', regex=False)
    # Convert 'nan' strings back to actual NaNs or handle them during float conversion if necessary
    # pd.to_numeric with errors='coerce' is robust for handling non-numeric strings resulting from
'nan'
    df_card_v1[col] = pd.to_numeric(df_card_v1[col], errors='coerce')
```

All datetime/time series data should be time series type (not string)

```
# 4. Convert datetime columns to datetime objects
date_cols = ['expires', 'acct_open_date']
for col in date_cols:
    df_card_v1[col] = pd.to_datetime(df_card_v1[col], format='%m/%Y')
```

Before

#	Column	Non-Null Count	Dtype
0	id	5599 non-null	int64
1	client_id	5599 non-null	int64
2	card_brand	5599 non-null	object
3	card_number	5599 non-null	int64
4	expires	5599 non-null	object
5	cvv	5599 non-null	int64
6	credit_limit	5587 non-null	object
7	acct_open_date	5599 non-null	object
8	year_pin_last_changed	5599 non-null	int64
9	days_since_last_trx	5599 non-null	int64
10	count_nonfraud_trx_L6M	3707 non-null	float64
11	amt_nonfraud_trx_L6M	3707 non-null	object
12	count_fraud_trx_L6M	547 non-null	float64
13	amt_fraud_trx_L6M	547 non-null	object

After

#	Column	Non-Null Count	Dtype
0	id	5599 non-null	object
1	client_id	5599 non-null	object
2	card_brand	5599 non-null	object
3	card_number	5599 non-null	object
4	expires	5599 non-null	datetime64[ns]
5	cvv	5599 non-null	object
6	credit_limit	5587 non-null	float64
7	acct_open_date	5599 non-null	datetime64[ns]
8	year_pin_last_changed	5599 non-null	int64
9	days_since_last_trx	5599 non-null	int64
10	count_nonfraud_trx_L6M	3707 non-null	Int64
11	amt_nonfraud_trx_L6M	3707 non-null	float64
12	count_fraud_trx_L6M	547 non-null	Int64
13	amt_fraud_trx_L6M	547 non-null	float64

4b. Take a look at the unique values in each column and see some typo data that either needs to be fixed or excluded from your analysis.

```
# Inspect unique values in 'card_brand' to identify
potential typos
print(df_card_v1['card_brand'].value_counts())
```

```
card_brand
Mastercard    2826
Visa          2162
Amex           402
JCB            206
Jcb              3
Name: count, dtype: int64
```

```python
# Remove leading/trailing whitespace to fix potential 'Visa
' vs 'Visa' duplicates
df_card_v1['card_brand'] =
df_card_v1['card_brand'].str.strip()

# Standardize 'Jcb' to 'JCB'
df_card_v1['card_brand'] =
df_card_v1['card_brand'].replace({'Jcb': 'JCB'})

# Verify the changes
print(df_card_v1['card_brand'].value_counts())
```

```
card_brand
Mastercard    2826
Visa          2162
Amex           402
JCB            209
Name: count, dtype: int64
```

if you find any duplicates, please delete the data. Please do elaborate on why you strongly suggested deleting the duplicated data and not replacing it with any other value.

```python
# Calculate the number of duplicate rows
duplicate_count = df_card_v1.duplicated().sum()
print(f'Number of duplicate rows: {duplicate_count}')

# Calculate the number of missing values for each column
missing_values = df_card_v1.isnull().sum()
print('\nMissing values per column:')
print(missing_values)

# Display the shape of the DataFrame
print(f'\nDataFrame shape: {df_card_v1.shape}')
```

```
Number of duplicate rows: 31

Missing values per column:
id                          0
client_id                   0
card_brand                  0
card_number                 0
expires                     0
cvv                         0
credit_limit               12
acct_open_date              0
year_pin_last_changed       0
days_since_last_trx         0
count_nonfraud_trx_L6M   1892
amt_nonfraud_trx_L6M     1892
count_fraud_trx_L6M      5052
amt_fraud_trx_L6M        5052
dtype: int64

DataFrame shape: (5599, 14)
```

```python
duplicate_mask = df_card_v1.duplicated()
duplicate_count = duplicate_mask.sum()

print(f"Number of duplicate rows found in df_card_v1:
{duplicate_count}")

if duplicate_count > 0:
    df_card_v1 =
df_card_v1[~duplicate_mask].reset_index(drop=True)
    print("Duplicate rows removed.")
else:
    print("No duplicate rows found.")

print(f"Shape of df_card_v1 after handling duplicates:
{df_card_v1.shape}")
```

```
Number of duplicate rows found in df_card_v1: 31
Duplicate rows removed.
Shape of df_card_v1 after handling duplicates: (5568, 14)
```

4d. Based on the card's expiry date, remove all cards that have already expired AND also exclude cards with a credit limit 0, data is calculated as of May 31, 2025.

```python
# Print shape before filtering
print(f'Shape before filtering: {df_card_v1.shape}')

# Filter out expired cards (keep cards expiring on or after
2025-05-01)
# Note: 'expires' is already datetime64 type
df_card_v1 = df_card_v1[df_card_v1['expires'] >= '2025-05-01']

# Filter out cards with 0 credit limit
df_card_v1 = df_card_v1[df_card_v1['credit_limit'] != 0]

# Print shape after filtering
print(f'Shape after filtering: {df_card_v1.shape}')

# Verify filtering criteria
print(f"Minimum expiration date: {df_card_v1['expires'].min()}")
print(f"Minimum credit limit: {df_card_v1['credit_limit'].min()}")
```

```
Shape before filtering: (5568, 14)
Shape after filtering: (5531, 14)
Minimum expiration date: 2025-05-01 00:00:00
Minimum credit limit: 301000.0
```

```python
# 1. Convert identifier column 'id' to string
df_user_v1['id'] = df_user_v1['id'].astype(str)
```

```python
# 2. Clean money columns: remove 'Rp' and '.', then convert to
float
user_money_cols = ['per_capita_income', 'yearly_income',
'total_debt']
for col in user_money_cols:
    # Remove 'Rp' and '.'
    df_user_v1[col] =
df_user_v1[col].astype(str).str.replace('Rp', '',
regex=False).str.replace('.', '', regex=False)
    # Convert to numeric
    df_user_v1[col] = pd.to_numeric(df_user_v1[col],
errors='coerce')
```

```python
# 3. Convert 'birthdate' to datetime
df_user_v1['birthdate'] =
pd.to_datetime(df_user_v1['birthdate'], format='%Y-%m-%d')
```

5a. Similar to step 4a, check the data types of each column and convert them to the correct types.

5b. Add age, retired flag, and DTI columns by using these logics:

```
analysis_cutoff_date = pd.to_datetime('2025-05-31')
df_user_v1['age'] = ((analysis_cutoff_date - df_user_v1['birthdate']).dt.days /
365.25).astype(int)

print("Calculated 'age' column and added to df_user_v1.")
df_user_v1[['birthdate', 'age']].head()
```

```
df_user_v1['retired_flag'] = False
df_user_v1.loc[df_user_v1['age'] >= df_user_v1['retirement_age'], 'retired_flag'] =
True

print("Calculated 'retired_flag' column using .loc and added to df_user_v1.")
df_user_v1[['age', 'retirement_age', 'retired_flag']].head()
```

```
df_user_v1['DTI'] = df_user_v1['total_debt'] / df_user_v1['yearly_income']
df_user_v1['DTI'] = df_user_v1['DTI'].replace([np.inf, -np.inf], np.nan).fillna(0)

print("Calculated 'DTI' column and handled division by zero.")
df_user_v1[['total_debt', 'yearly_income', 'DTI']].head()
```

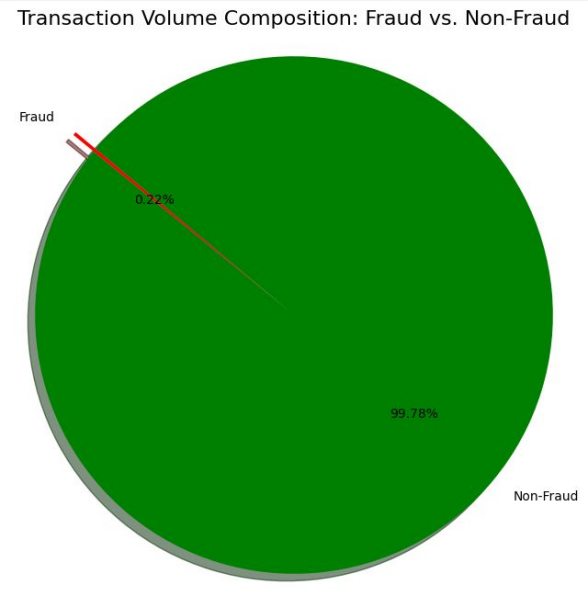
a. Calculate the total net profit from all transactions in the last 6 months.

```python
# 1. Calculate sum of non-fraud transactions
total_non_fraud =
df_card_v1['amt_nonfraud_trx_L6M'].sum()

# 2. Calculate sum of fraud transactions
total_fraud = df_card_v1['amt_fraud_trx_L6M'].sum()

# 3. Compute total volume
total_volume = total_non_fraud + total_fraud

# 4. Calculate fraud rate percentage
fraud_rate = (total_fraud / total_volume) * 100

# 5. Print the values with descriptive labels
print(f"Total Transaction Volume: {total_volume:,.2f}")
print(f"Total Fraud Amount: {total_fraud:,.2f}")
print(f"Fraud Rate: {fraud_rate:.2f}%")
```

Total Transaction Volume: 457,069,876,900.00
Total Fraud Amount: 1,013,378,300.00
Fraud Rate: 0.22%

b. Calculate the fraud rate of RevoBank. Fraud rate is the percentage of fraud amount from overall transaction volume. Do you think the fraud rate is still considered safe to RevoBank?

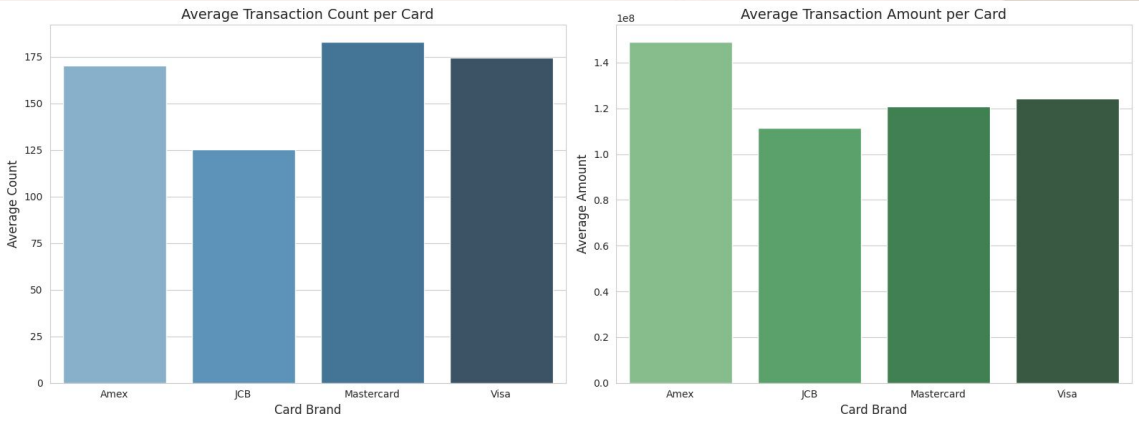
```
import matplotlib.pyplotas plt

# Data for the pie chart
labels = ['Fraud', 'Non-Fraud']
sizes = [total_fraud, total_non_fraud]
colors = ['red', 'green']
explode = (0.1, 0)  # explode the 1st slice (Fraud)

# Create the pie chart
plt.figure(figsize=(8, 8))
plt.pie(sizes, explode=explode, labels=labels,
colors=colors, autopct='%1.2f%%',
        shadow=True, startangle=140)
plt.title('Transaction Volume Composition: Fraud
vs. Non-Fraud', fontsize=16)
plt.axis('equal')  # Equal aspect ratio ensures
that pie is drawn as a circle.

plt.show()
```



c. Is there any difference in transaction behavior per card brand? Present your findings in a data visualisation.

```
# Group by 'card_brand' and calculate averages for
non-fraud transactions
brand_avg_metrics =
df_card_v1.groupby('card_brand')[['count_nonfraud_tr
x_L6M',
'amt_nonfraud_trx_L6M']].mean().reset_index()

# Display the resulting DataFrame
print(brand_avg_metrics)
```

```
import matplotlib.pyplot  as plt
import seaborn  as sns

# Create a figure with two subplots side-by-side
fig, axes = plt.subplots (1, 2, figsize= (16, 6))

# 1. Bar chart for Average Transaction Count
sns.barplot (x='card_brand' , y='count_nonfraud_trx_L6M'  ,
data=brand_avg_metrics  , ax=axes [0], palette= 'Blues_d' ,
hue='card_brand' , legend= False )
axes [0].set_title ('Average Transaction Count per Card'  , fontsize= 14)
axes [0].set_xlabel ('Card Brand' , fontsize= 12)
axes [0].set_ylabel ('Average Count' , fontsize= 12)

# 2. Bar chart for Average Transaction Amount
sns.barplot (x='card_brand' , y='amt_nonfraud_trx_L6M'  ,
data=brand_avg_metrics  , ax=axes [1], palette= 'Greens_d' ,
hue='card_brand' , legend= False )
axes [1].set_title ('Average Transaction Amount per Card'  , fontsize= 14)
axes [1].set_xlabel ('Card Brand' , fontsize= 12)
axes [1].set_ylabel ('Average Amount' , fontsize= 12)

# Adjust layout to prevent overlap
plt.tight_layout ()
plt.show ()
```

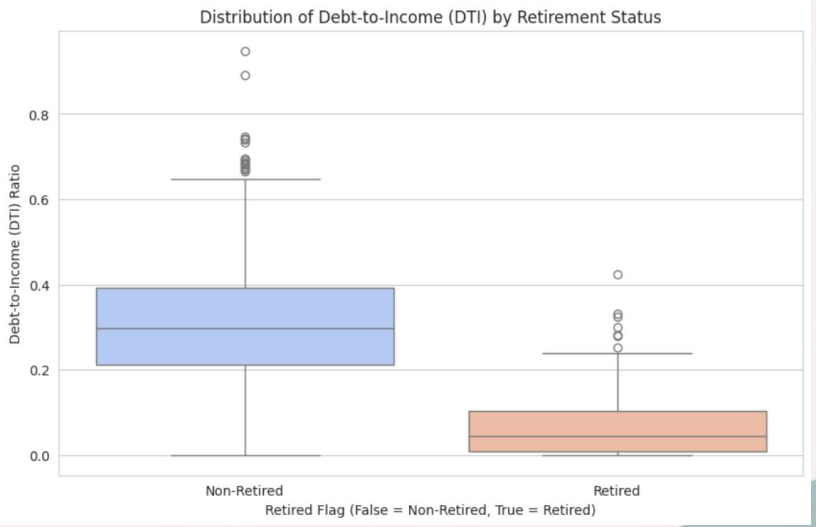


Mastercard exhibits the highest frequency of usage with an average of **~122** non-fraud transactions over the last 6 months. However, **Amex** stands out for high-value usage, having the highest average transaction amount of approximately **Rp98.8 Million**. **JCB** records the lowest figures for both transaction frequency and average amount.

d. Compare the debt-to-income ratios of retired vs non-retired users. The debt-to-income ratio definition is the total debt per yearly income. Are retired users riskier?

```
dti_stats_by_retired_flag =
df_user_v1.groupby ('retired_flag' )['DTI'].descri
be(
    percentiles= [.25, .50, .75]
)
print ("Descriptive statistics of DTI by
retired_flag:" )
display (dti_stats_by_retired_flag )
```

```
sns.set_style("whitegrid")

plt.figure(figsize=(10, 6))
sns.boxplot(x='retired_flag', y='DTI', data=df_user_v1,
hue='retired_flag', palette='coolwarm', legend=False)
plt.title('Distribution of Debt-to-Income (DTI) by
Retirement Status)
plt.xlabel('Retired Flag (False = Non-Retired, True =
Retired) ')
plt.ylabel('Debt-to-Income (DTI) Ratio)'
plt.xticks([0, 1], ['Non-Retired', 'Retired'])
plt.show()
```

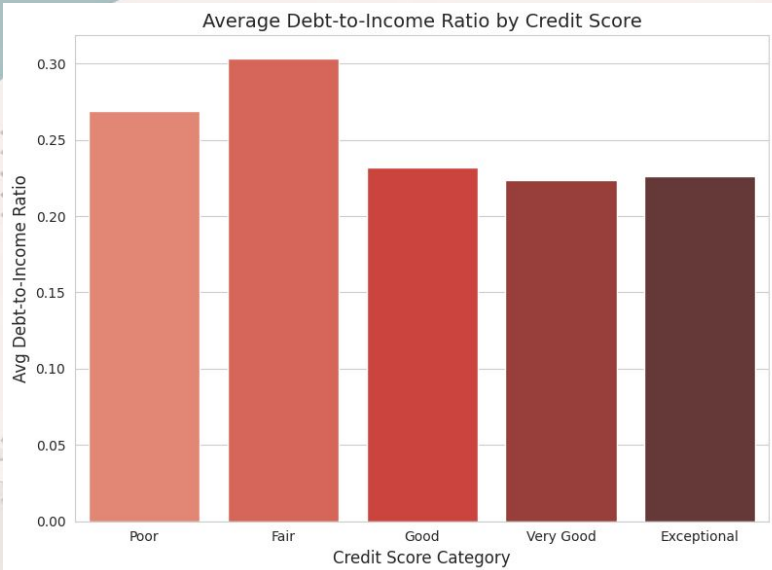


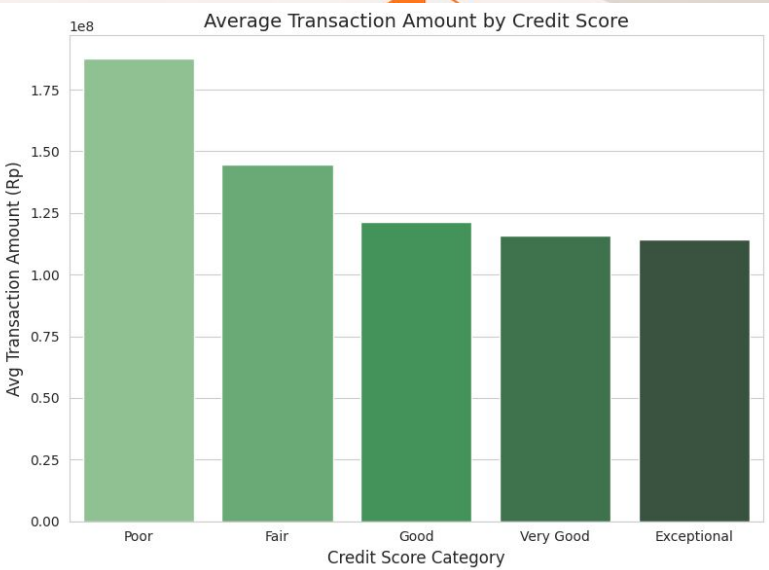
The boxplot and statistics confirm that retired users generally carry very little debt relative to their income, indicating a conservative financial approach or paid-off major liabilities (like mortgages).

e. Calculate the average of Debt-to-Income ratio (DTI) based on Credit Score Category (Poor, Fiar, Good, Very Good, Exceptional). Does the DTI align with the credit score? You can also see the transaction behaviour of each credit score category. Does higher DTI give higher transactions to RevoBank? If so, do you think that is riskier? Provide recommendations and present your findings in a data visualisation.

```
# Define FICO score bins and labels
# Bins: Poor (300-579), Fair (580-669), Good (670-739), Very Good (740-799),
Exceptional (800-850)
fico_bins = [299, 579, 669, 739, 799, 850]
fico_labels = ['Poor', 'Fair', 'Good', 'Very Good', 'Exceptional']

# Create 'credit_score_category' column
df_merged['credit_score_category'] = pd.cut(df_merged['credit_score'],
bins=fico_bins, labels=fico_labels)

# Verify the categorization by displaying value counts
print(df_merged['credit_score_category'].value_counts().sort_index())
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Create a figure with two subplots side-by-side
fig, axes = plt.subplots (1, 2, figsize= (16, 6))

# 1. Bar chart for Average Debt-to-Income Ratio
sns.barplot (x='credit_score_category' , y='DTI', data=fico_stats , ax=axes [0], palette= 'Reds_d' , hue='credit_score_category' ,
legend= False)
axes [0].set_title ('Average Debt-to-Income Ratio by Credit Score' , fontsize= 14)
axes [0].set_xlabel ('Credit Score Category' , fontsize= 12)
axes [0].set_ylabel ('Avg Debt-to-Income Ratio' , fontsize= 12)

# 2. Bar chart for Average Transaction Amount
sns.barplot (x='credit_score_category' , y='amt_nonfraud_trx_L6M' , data=fico_stats , ax=axes [1], palette= 'Greens_d' ,
hue='credit_score_category' , legend= False)
axes [1].set_title ('Average Transaction Amount by Credit Score' , fontsize= 14)
axes [1].set_xlabel ('Credit Score Category' , fontsize= 12)
axes [1].set_ylabel ('Avg Transaction Amount (Rp)' , fontsize= 12)

# Adjust layout to prevent overlap
plt.tight_layout ()
plt.show ()
```



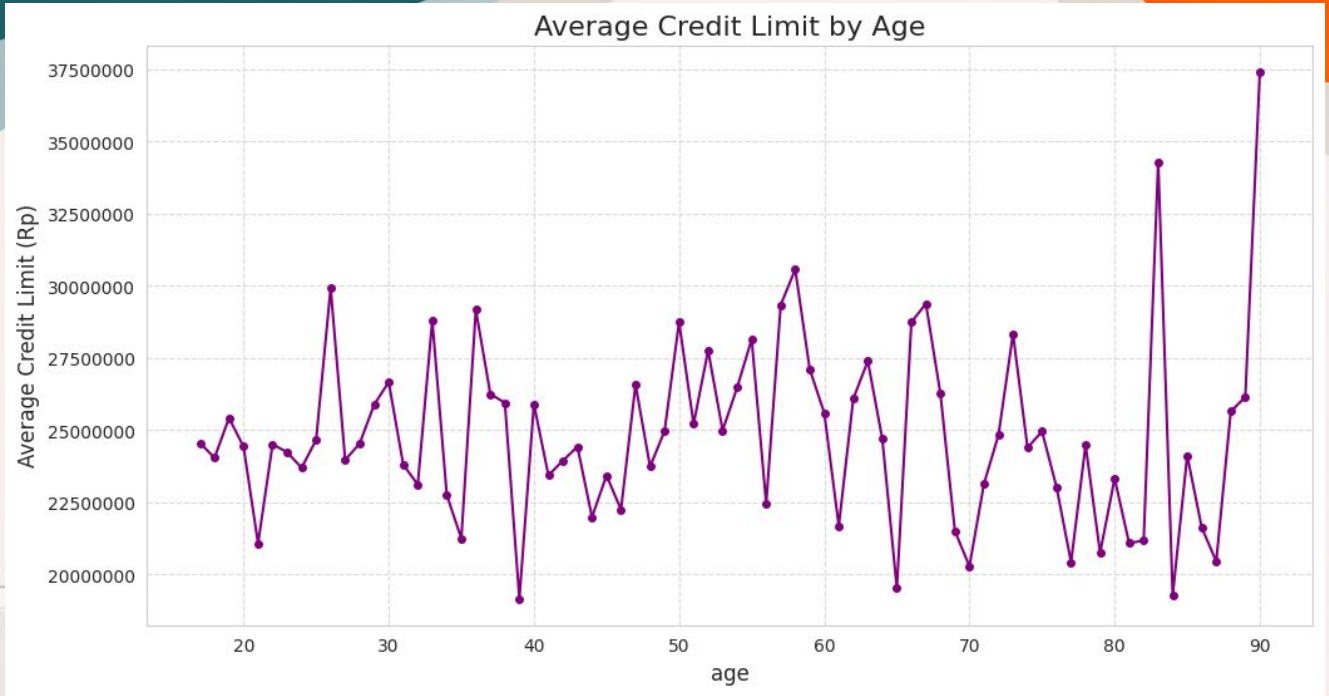


- **Debt-to-Income Trends:** Financial leverage generally decreases as credit score improves. The 'Fair' category is the most leveraged (0.30), while 'Good', 'Very Good', and 'Exceptional' categories stabilize around a lower ratio of 0.22-0.23.

f. Is there a relationship between user age and credit limit? If so, what is your recommendation for the bank to improve credit card usage? Present your findings in a line chart.

```
# Filter for ages <= 90
df_filtered_age =
df_merged[df_merged['age'] <= 90]

# Group by 'Age' and calculate mean
'credit_limit'
age_credit_limit =
df_filtered_age.groupby('age')['credit_lim
it'].mean().reset_index()

# Display the first few rows
print(age_credit_limit.head())
```

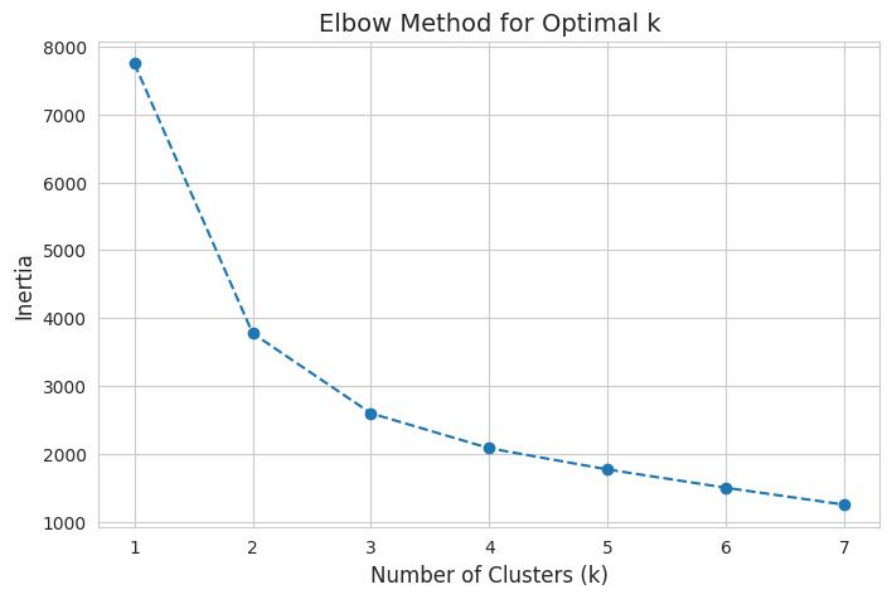
```
import matplotlib.pyplotas plt

# Create a line chart for Average Credit Limit by Age
plt.figure(figsize=(12, 6))
plt.plot(age_credit_limit['age'],
age_credit_limit['credit_limit'], color='purple',
marker='o', markersize=4, linestyle='-')

# Add title and labels
plt.title('Average Credit Limit by Age', fontsize=16)
plt.xlabel('age', fontsize=12)
plt.ylabel('Average Credit Limit (Rp)', fontsize=12)

# Format y-axis to standard numbers
plt.ticklabel_format(style='plain', axis='y')

plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



the analysis suggests that Age is a secondary factor. There is no exponential growth in credit limits based on age alone. Profitability is likely better served by targeting Usage Frequency and Income Levels, as these factors show stronger correlations with value than age does.

```python
# Group by 'client_id' and aggregate features
# Recency: min(days_since_last_trx), Frequency:
sum(count_nonfraud_trx_L6M),
# Monetary: sum(amt_nonfraud_trx_L6M), Income:
max(yearly_income)
df_customer = df_merged.groupby('client_id').agg({
    'days_since_last_trx': 'min',
    'count_nonfraud_trx_L6M': 'sum',
    'amt_nonfraud_trx_L6M': 'sum',
    'yearly_income': 'max'
}).reset_index()

# Rename columns for clarity
df_customer.rename(columns={
    'days_since_last_trx': 'Recency',
    'count_nonfraud_trx_L6M': 'Frequency',
    'amt_nonfraud_trx_L6M': 'Monetary',
    'yearly_income': 'Income'
}, inplace=True)

# Display the first few rows of the customer profile DataFrame
print(df_customer.head())
```

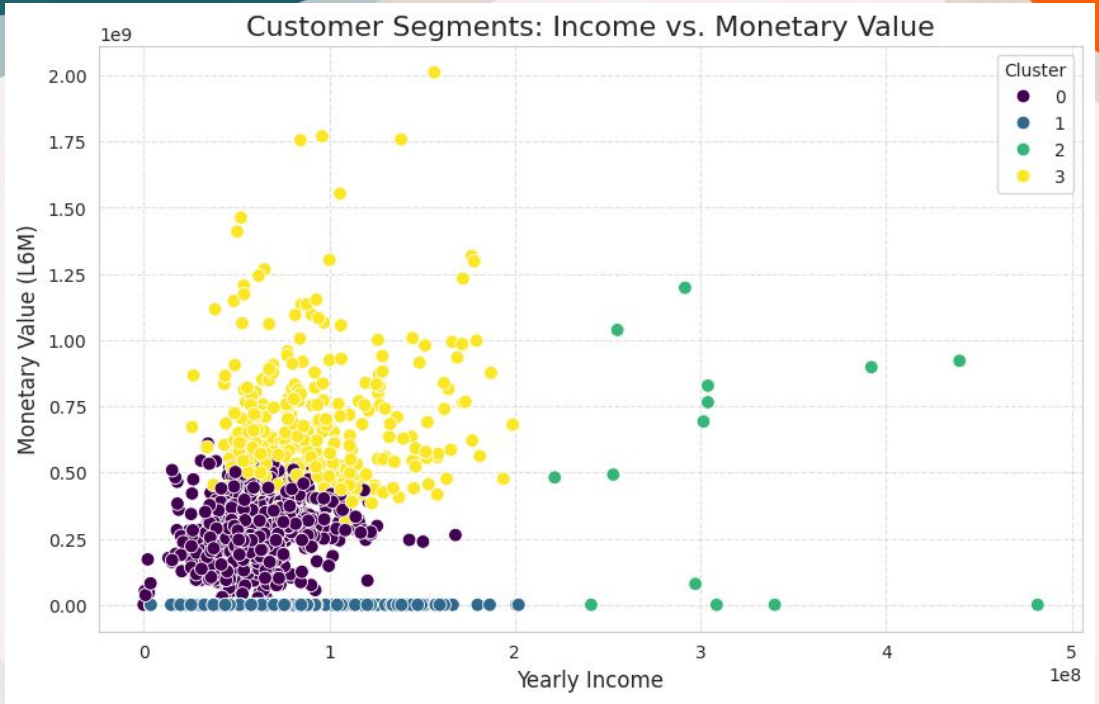
```python
# Select features for clustering
features = ['Recency', 'Frequency', 'Monetary', 'Income']
X = df_customer[features]

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Use the Elbow Method to find optimal k
inertia = []
k_range = range(1, 8)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot the Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o', linestyle='--')
plt.title('Elbow Method for Optimal k', fontsize=14)
plt.xlabel('Number of Clusters (k)', fontsize=12)
plt.ylabel('Inertia', fontsize=12)
plt.xticks(k_range)
plt.grid(True)
plt.show()
```



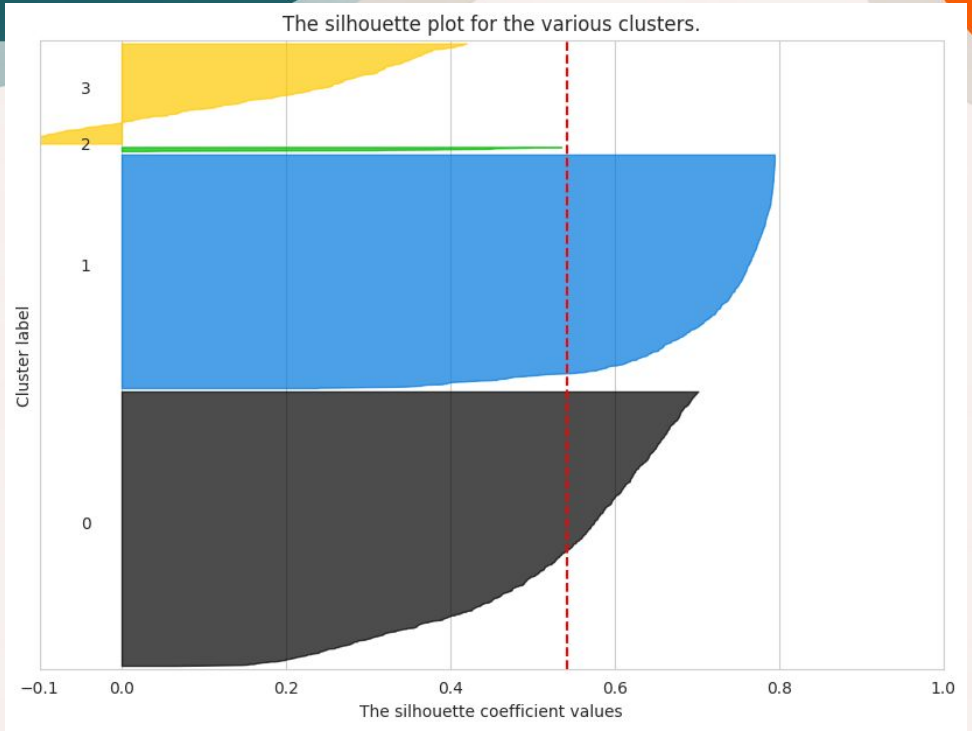
Based on the Elbow Method (significant drop in inertia up to k=3 or k=4), I will choose k=4 to identify distinct customer segments. I will apply K-Means, assign cluster labels, and calculate the average feature values for each cluster to profile them.

```python
k = 4
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df_customer['Cluster'] = kmeans.fit_predict(X_scaled)

# Calculate average values for each cluster to interpret segments
cluster_summary =
df_customer.groupby('Cluster')[features].mean().reset_index()

# Display the cluster summary
print(cluster_summary)
```

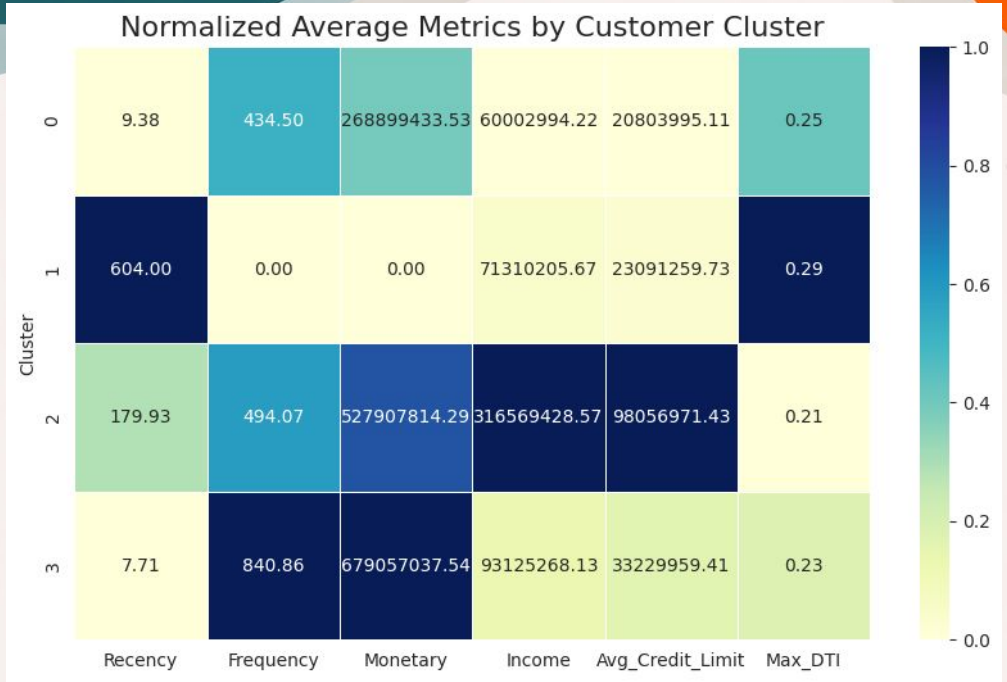
```python
# Visualize segments: Income vs. Monetary
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Income', y='Monetary', hue='Cluster',
data=df_customer, palette='viridis', s=60)

# Highlight centroids if needed, or just rely on the scatter points
plt.title('Customer Segments: Income vs. Monetary Value', fontsize=16)
plt.xlabel('Yearly Income', fontsize=12)
plt.ylabel('Monetary Value (L6M)', fontsize=12)
plt.legend(title='Cluster')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Using K-Means clustering (k=4), the analysis identified four distinct segments. The **"High Potential"** segment is **Cluster 2**, characterized by the **highest average yearly income (~Rp 316 Million)** but lower transaction volume and spending compared to the top-performing group. This indicates that these customers have the financial capacity to spend more but are currently underutilizing RevoBank services.



While k=2 yielded the highest mathematical score (0.5638), the score for k=4 (0.5423) is very comparable and indicates a strong structure. From a business perspective, 4 clusters provide significantly more actionable granularity (distinguishing "High Potential" from "Champions" and "Mass Market") than a simple binary split. The silhouette plot for k=4 confirms that the clusters are well-defined, with most samples exceeding the average silhouette score, validating the robustness of this segmentation model.



- **High Value (Cluster 3 - "Champions"):** This segment drives the business, generating the highest transaction volume (**~841**) and monetary value (**~Rp 679M**) while maintaining high activity levels.
- **High Potential (Cluster 2 - "Wealthy"):** This group has the strongest financial profile with the highest **Average Income (~Rp 316M)** and **Credit Limit (~Rp 98M)**, coupled with the **lowest Debt-to-Income ratio**, indicating significant untapped spending power.
- **High Risk (Cluster 1):** These customers are inactive (churned with Recency **~604 days**) and exhibit the **highest Debt-to-Income ratio**, posing a potential credit risk.